

Trie :

高效地存储和查找字符串集合的数据结构

例：存储

a b c d e f

a b d e f

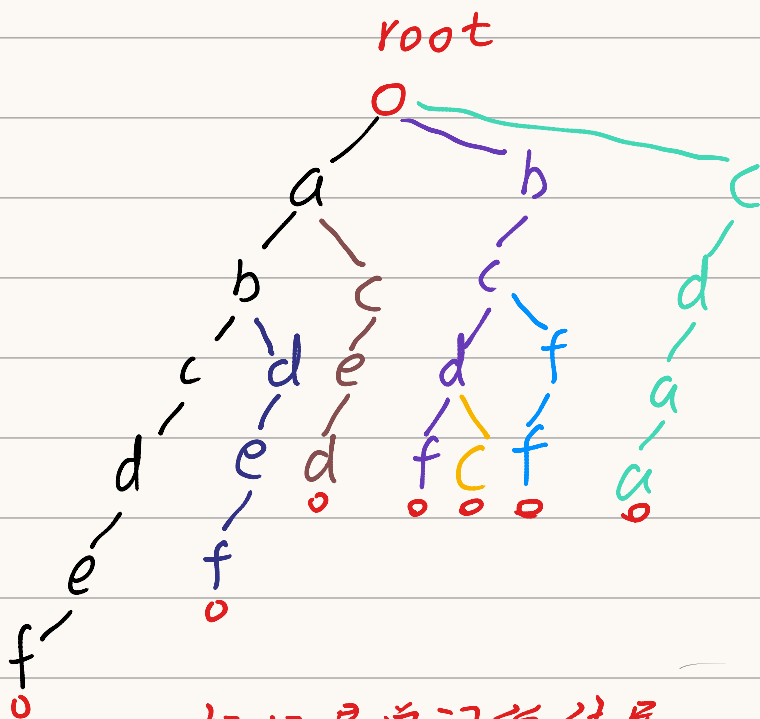
a c e d

b c d f

b c f f

1 d a a

b c d c



0: 标记是单词的结尾

查找: a c e d 有这条树枝且 d 有标记 ✓

$a \ b \ c \ f$ 有 $a \ b \ c$ 之后无 f x

$a b c d$ 有 $a b c d$ 但 d 无标记 $\times$

存储式：

(26个手写英文字母)

son[N][26] 每个结点至多 26 个子树

$cnt[N]$ 以当前结点结尾的单词有多少个

idx 当前存储到哪一位

下标是0的点, 既是根结点, 又是空结点

作 root 也, 作 Null

存: str

遍历 str (i) 把当前的 i 映射为对应序号 (1~26)

int p=0 定位当前在 trie 树的位置

for (int i=0; str[i]; i++)

int u = str[i] - 'a' 0~25

if (!son[p][u]) son[p][u] = ++idx

p = son[p][u] p 转到 str[i] 在 trie 树的位置, 为下一轮准备

cnt[p]++

查询

int p = 0

for (int i=0; str[i]; i++)

int u = str[i] - 'a'

if (!son[p][u]) return 0 不存在

p = son[p][u]

return cnt[p]

补充: son[N][26] 中一维 N 指的是总节点数, 二维存的是结点关系 (谁是谁儿子)

[0][1] = 3 [0] 表示根结点 [1] 对应 'b'

表示根结点有 1 个儿子 'b', 其下标为 3

[3][2] = 8 表示根结点的儿子 'b' 也有一个儿子 'c', 下标为 8

root 0 1 2 3 4 5 6 7 8 查 bc p: 0 → 3 → 8

若想存汉字,我们可以开 `son[N][2]`

将汉字转成对应的二进制编码

增加树的深度而不是宽度

835. Trie字符串统计

题目

提交记录

讨论

题解

视频讲解

维护一个字符串集合，支持两种操作：
1. `I x` 向集合中插入一个字符串 `x`；
2. `Q x` 询问一个字符串在集合中出现了多少次。
共有 N 个操作，所有输入的字符串总长度不超过 10^5 ，字符串仅包含小写英文字母。
输入格式
第一行包含整数 N ，表示操作数。
接下来 N 行，每行包含一个操作指令，指令为 `I x` 或 `Q x` 中的一种。
输出格式
对于每个询问指令 `Q x`，都要输出一个整数作为结果，表示 `x` 在集合中出现的次数。
每个结果占一行。
数据范围
 $1 \leq N \leq 2 \times 10^4$
输入样例：
5
I abc
Q abc
Q ab
I ab
Q ab
输出样例：
1
0
1

难度：

简单

时空限制：1s / 64MB

总通过数：126231

总尝试数：183468

来源：

模板题

算法标签

```
1 #include <stdio.h>
2
3 const int N = 1e5 + 10;
4 // 注意，这里N的大小应该是所有字符串的总长度
5 // 单条字符串最大长度*总操作次数
6 int n;
7 int SON[N][26], cnt[N];
8 int idx;
9 // 我们的0号位既作为root也作为NULL
10 // 因此有效值是从1开始的，应该是 ++ idx
11 void insert(char *exp) {
12     int p = 0;
13     for(int i = 0; exp[i]; i++) {
14         int num = exp[i] - 'a';
15         if(SON[p][num] == 0) SON[p][num] = ++ idx;
16         // 这里注意，只有当前路径不存在的时候我们才需要创建新的
17         p = SON[p][num];
18     }
19     cnt[p] ++;
20     return ;
21 }
22
23 int find(char *exp) {
24     int p = 0;
25     for(int i = 0; exp[i]; i++) {
26         int num = exp[i] - 'a';
27         if(SON[p][num] == 0) return 0;
28         // 不存在的时候直接return 而不是 break!
29         p = SON[p][num];
30     }
31     return cnt[p];
32 }
33
34 int main() {
35     scanf("%d", &n);
36     char exp[100000 + 10], choose[5];
37     for(int i = 0; i < n; i++) {
38         scanf("%s", choose);
39         scanf("%s", exp);
40
41         if(choose[0] == 'I') insert(exp);
42         else printf("%d\n", find(exp));
43     }
44
45     return 0;
46 }
```

143. 最大异或对

🏠 题目

📝 提交记录

💬 讨论

📖 题解

📺 视频讲解

在给定的 N 个整数 $A_1, A_2, \dots, A_N$ 中选出两个进行 xor (异或) 运算, 得到的结果最大是多少?

输入格式

第一行输入一个整数 N 。

第二行输入 N 个整数 $A_1 \sim A_N$ 。

输出格式

输出一个整数表示答案。

数据范围

$1 \leq N \leq 10^5$,
 $0 \leq A_i < 2^{31}$

输入样例:

```
3
1 2 3
```

输出样例:

```
3
```

难度:

简单

时空限制:

1s / 64MB

总通过数:

87438

总尝试数:

173223

来源:

《算法竞赛进阶指南》

模板题

《信息学奥赛一本通》算法提高篇

算法标签

```
1 #include <stdio.h>
2 #include <math.h>
3
4
5 /*
6 思路: 将待存储的数以二进制形式从高位到低位存储
7 统一存储31位 (高位不足的补0)
8 全部数据存储结束后, 再遍历每个数
9 异或: 0 1 -> 1 else -> 0
10 从高位开始的原因: 高位的影响最大
11 0~x-1位即使全部为异或1, 值仍小于x异或为1的结果
12
13 我们从高位开始, 对于当前数, 如果存储的所有数据中这一位
14 有和当前位不一样的, 那么就走这条道
15 SON[p][1 - u] != 0, 这一位满足异或为1, 记录到tmp中
16 否则, 就走 SON[p][u], 这一位异或为0, tmp不更新
17 (因为我们是录入了完整的31位, 所以对于任一结点, 子结点要么有要么有1要么有0
18 一定可以继续往下走的)
19 )
20
21 维护一个全局MAX, 时刻更新
22 */
23 const int N = 1e5 * 32 + 10;
24 int n;
25 int SON[N][2];
26 int idx;
27 int MAX = 0;
28 int nums[100010];
29
30 void insert(int x) {
31     int bin_num[31];
32     for(int i = 30; i >= 0; i--) {
33         bin_num[i] = (x >> (30 - i)) & 1;
34     } // 从高位到低存储31位二进制表达
35     // 这里用 (x >> (30 - i)) & 1, 不用修改x的值就能获取每一位
36
37     int p = 0; // 存入到trie树中
38     for(int i = 0; i < 31; i++) { // 从高位开始存
39         if(SON[p][bin_num[i]] == 0) SON[p][bin_num[i]] = ++ idx;
40         p = SON[p][bin_num[i]];
41     }
42 }
43
44 int getRes(int cnt) {
45     // 这里不使用pow来获取2的k次
46     // 因为pow是浮点数运算, 效率很低很低
47     int res = 1;
48     for(int i = 0; i < cnt; i++) {
49         res *= 2;
50     }
51     return res;
52 }
53
```

```
54 int main() {
55     scanf("%d", &n);
56     for(int i = 0; i < n; i++) {
57         scanf("%d", &nums[i]);
58         insert(nums[i]); // 读入数据, 存入trie树
59     }
60
61     for(int i = 0; i < n; i++) {
62         int tmp = 0; // 临时tmp
63
64         int bin_num[31];
65         for(int j = 30; j >= 0; j--) {
66             bin_num[j] = (nums[i] >> (30 - j)) & 1;
67         } // 获取当前这个数的31位
68
69         // 从最高位开始, 当前位为u, 不断尝试走1-u这条路
70         int p = 0;
71         for(int j = 0; j < 31; j++) {
72             int u = bin_num[j];
73             if(SON[p][1 - u] != 0) {
74                 tmp += getRes(30 - j);
75                 p = SON[p][1 - u];
76             } else {
77                 p = SON[p][u];
78             }
79         }
80         if(tmp > MAX) MAX = tmp; // 维护MAX
81     }
82     printf("%d", MAX);
83
84
85     return 0;
86 }
```